



SPENDWISE - SOLUTION REQUIREMENTS

Version: 1.0.0 | Date: March 7, 2026 | Status: ✔ All Requirements Met

1. FUNCTIONAL REQUIREMENTS (IMPLEMENTED)



FR-01: User Authentication

ID	Requirement	Implementation	Status
FR-01.1	User Registration	POST /api/auth/register	✔
FR-01.2	User Login	POST /api/auth/login	✔
FR-01.3	JWT Token Generation	jwt.sign with 30d expiry	✔
FR-01.4	Password Hashing	bcrypt with salt 10	✔
FR-01.5	User Profile	GET /api/auth/profile	✔

POST /api/auth/register

Request: {"name": "Test User", "email": "test@email.com", "password": "123456"}
Response: {"_id": "...", "name": "Test User", "email": "...", "token": "JWT_TOKEN"}

POST /api/auth/login

Request: {"email": "test@email.com", "password": "123456"}
Response: {"_id": "...", "name": "Test User", "email": "...", "token": "JWT_TOKEN"}



FR-02: Transaction Management

ID	Requirement	Implementation	Status
FR-02.1	Create Transaction	POST /api/transactions	✔
FR-02.2	Get All Transactions	GET /api/transactions	✔

FR-02.3	Get Single Transaction	GET /api/transactions/:id	✓
FR-02.4	Update Transaction	PUT /api/transactions/:id	✓
FR-02.5	Delete Transaction	DELETE /api/transactions/:id	✓

Transaction Categories Implemented:

- Food
- Transport
- Shopping
- Entertainment
- Bills
- Salary
- Health
- Education
- Other

POST /api/transactions

Request: {"title": "Lunch", "amount": 25.50, "type": "expense", "category": "Food"}

Response: {"_id": "...", "title": "Lunch", "amount": 25.50, "type": "expense", ...}

FR-03: Analytics & Reporting

ID	Requirement	Implementation	Status
FR-03.1	Transaction Statistics	GET /api/transactions/stats/summary	✓
FR-03.2	Income vs Expense	Server-side aggregation	✓
FR-03.3	Category Breakdown	Aggregated by category	✓
FR-03.4	Monthly Summary	Last 6 months trend	✓
FR-03.5	Balance Calculation	Real-time updates	✓

GET /api/transactions/stats/summary

Response: { "summary": {"totalIncome": 3000, "totalExpense": 125.50, "balance": 2874.50}, "categoryBreakdown": {"Food": 125.50}, "monthlySummary": [...] }

2. DATA MODELS (IMPLEMENTED)

User Model - models/User.js

```
const userSchema = new mongoose.Schema({ name: { type: String, required: true, trim: true }, email: { type: String, required: true, unique: true, lowercase: true }, password: { type: String, required: true, minlength: 6, select: false }, }, { timestamps: true }); // Password hashing middleware userSchema.pre('save', async function(next) { if (!this.isModified('password')) return next(); this.password = await bcrypt.hash(this.password, 10); next(); }); // Password comparison method userSchema.methods.comparePassword = async function(password) { return await bcrypt.compare(password, this.password); };
```

Transaction Model - models/Transaction.js

```
const transactionSchema = new mongoose.Schema({ user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }, title: { type: String, required: true, trim: true }, amount: { type: Number, required: true }, type: { type: String, enum: ['income', 'expense'], required: true }, category: { type: String, required: true }, description: { type: String, trim: true }, date: { type: Date, default: Date.now }, }, { timestamps: true });
```

3. API ENDPOINTS SUMMARY

Method	Endpoint	Description	Auth	Status
GET	/	API info	No	✓
POST	/api/auth/register	Register user	No	✓
POST	/api/auth/login	Login user	No	✓
GET	/api/auth/profile	Get profile	Yes	✓
GET	/api/transactions	Get all transactions	Yes	✓
GET	/api/transactions/:id	Get single transaction	Yes	✓
POST	/api/transactions	Create transaction	Yes	✓
PUT	/api/transactions/:id	Update transaction	Yes	✓
DELETE	/api/transactions/:id	Delete transaction	Yes	✓
GET	/api/transactions/stats/summary	Get statistics	Yes	✓

Spendwise Solution Requirements Document | Version 1.0.0 | March 7, 2026